Requirements gathering

Elicitation Method:

Our requirement's gathering method was to interview barbers and customers one on one. This allowed us to gather a lot of high-quality information per interviewee. Our plan was to do 7 Barber interviews and 7 Customer interviews. This translates to each group member doing 2 interviews each.

We decided to do two different rounds of interviews. First, we would do the "initial" interviews. There is one initial interview per user type (barber, customer). The initial interviews carried out had the intention of getting a very basic idea of what we were looking at making. The second round of interviews was the "primary" round, this is where we got most of the data. The initial and primary rounds had different aims. Our motivation for the initial round was to get a basic understanding of what questions we should be asking for the primary rounds. Next, our motivation for the primary round was to get a much more detailed understanding of the user requirements. It was important that we did the initial round as that gave us some questions that we could ask later in the primary round.

Analysis:

Once all 14 interviews had been completed some trends were already obvious without any proper analysis having been undertaken. Before we even started the initial interviews, we each had our own idea about what the system would look like. Some of us thought that timetabling was the main feature, but we quickly discovered that the discover page was much more anticipated by both the barbers and customers we interviewed. The barber Maciek said "A useful thing the app could change or provide is advertisement and showcase skills and different skills that the barbers have and could provide.", this shows that barbers like the discover page but do not like the timetabling system, when asked if Maciek would change to our timetabling system they said "everyone has already familiarised themselves so it'd be kind of a hassle to change everything."

These patterns were identified before the actual analysis was even done, once the analysis was complete it was even more clear that we should focus on a social media style app for barbers to advertise their work and get customers. To analyse the transcripts generated by the recorded interviews we filled out a template with 4 sections, 1, things they like about the app, 2, things they don't like, 3, things they would add, 4, anything else worth note. We gave the template and transcript to a LLM (ChatGPT5) and asked them to fill out the template for each transcript. Different templates were used for the initial interview. The final analysis was an aggregated analysis of all the previous interview analyses. This gave us a very comprehensive understanding of our user's and their needs.

The main challenge we faced during this process was social anxiety. Walking into a barbershop and asking for a barber to give 10-15 minutes of their time for free and then having to talk to them for 10-15 minutes was stressful for those of us who did barber interviews. Some of us only

did customer interviews which were much easier since we could get away with interviewing one of our friends who weren't in the group.

User requirements

User requirement	Description
Customer requirements	
Discover page	User must see an “endless” scroll of photos posted by barbers. Customer can see barbers' distance and rating in small text atU bottom of image so they can easily see important information. Customer must be able to filter their feed to get a specific service that they want. Filter by Followed barbers, or filter by tags like “curly African hair” or “fair hair”. Barbers from interviews 1,2,3,4,6 and 7 support the implementation of this system as the "main feature”.
Barber profile	Customer must be able to see where barber is. See other haircuts performed by the barber and must be able to access the barber's website if there is one. Customer can see barbers rating and reviews. Customer can follow barber or share their profile via a URL link. Customer can see barber’s specialities. Customer can see when barber is working. Barbers from interviews 2,3,4 ike barber profile idea and barber from interview 4 mentioned using a link to barber website instead of in-built timetabling system.
Interactive map	A map where users can see barbershops and can click on the shops to be taken to that barbershop’s page None of the interviewees directly said anything about the interactive map, many customers mentioned wanting the app to be convenient and easy to use, we think that knowing where your barber is, is quite convenient.
Barbershop page	A page where user can see an overview of the barbershop. Can see all barbers that work there as well as general barbershop info. This is a page needed for the interactive map to work and thus is an extension of the “convenience” requitement.
Login page	Customers can login to their account or continue as guest. Customer can’t login without login page

Barber requirements	
Barber dashboard	This is where the barber manages their profile. They can make posts to the discover page. They can edit their profile information. They can specify their exact services and skills. They can also upload images for their profile (not discover page).

	For any of the apps features to work requires barbers participating with the app, they want to set it up and then that's it, no constantly updating wait times (1,2,3,4,6 all disapproved). Barbers are happy to upload photos to app (1,2,3,4,6 think its worth the time)
Login page	Barber must login to manage their dashboard. Barber can't login without login page.

System requirements (functional)

System requirement	Description	Priority
	Discover page	
User must see an “endless” scroll of photos posted by barbers.	This is the main feature of the app. An Instagram-like reel of images posted by barbers. User can scroll with their mouse (desktop) or swipe with their thumb (mobile) to scroll up and down their feed. This must be low effort, tiny thumb swipe = next image, tiny mouse scroll = next image.	High
Customer can see barbers' distance and rating in small text at bottom of image so they can easily see important information.	When a post is seen on the discover page, the customer can see the barber's distance from them in km and see the average star review left by other customers next to the icon at the bottom of the image. This makes it easier for customers to make decisions on which barber to choose.	Med
Customer must be able to filter their feed to get a specific service that they want.	At the top of the discover page will be a search bar and next to the search bar there will be generic filters drop down list. The drop-down list will contain generic filters (Proximity, Highest Rated, Most Reviews). The user can type in hair types or other tags into the search bar to find specific haircuts.	Low
Filter by followed barbers.	If customer is logged in (not on temporary guest account) they can filter their feed by the barbers, they have followed in past sessions. This filter will be in the drop-down list at top of page.	Low
Customer can click on image to be taken to barber’s profile.	If the customer likes the barber’s photo they can tap with their thumb or click with their mouse and then will be taken to that specific barber’s profile.	High
	Login and signup	
Sign-up process	When opening the app you can chose to “login” or “continue as guest”. If “login” is chosen, you can choose to login as barber or customer or sign-up. If “continue as guest” is chosen, you will be logged in as a customer with a temporary guest account. If “sign-up” is selected during the login form you’ll be asked whether you want to create a barber account or a customer account.	High

Barber account creation	The barber must enter their email, username and create a secure password.	High
Customer account creation	The customer must enter their email, username and create a secure password. They will also need to specify their location for in-app features to work.	High
Barber dashboard		
They can edit their profile information.	The system will provide an “Edit Profile” function that when selected displays an edit profile form containing the fields: name, username, location, services and website. The system will then allow barber users to save changes after the fields have been validated.	High
They can make posts to the discover page.	Barbers can create posts that can be seen by customers on the barber dashboard where an optional short description is also added before posting.	High
They can specify their exact services and skills.	Barbers will be able to put on their dashboards what services they can provide by clicking on an ‘add services’ button. This will be seen by customers and allow them to search for specific services.	Low
They can upload images for their profile	Barbers will be able to upload images of their work onto their profile by pressing an "upload to profile” button in the “edit profile” form, prompting them to select images to upload for their profile. Customers will be able to see these images when they click on a barber’s profile as a sort of preview into what their haircut from the barber will look like. The upload form will have a box that the customer can drag and drop PNG files into.	High
Interactive map		
Users can see barbershop pins on map and can click on the pins to be taken to that barbershop’s page	When the interactive map icon is selected the system will switch pages to a map containing only barbershops, when the pin is selected a pop up above the pin is shown with the name of the barber, profile picture and their star rating when the profile picture is selected the system redirects the user to the barbers profile.	Medium
User can specify their location.	In the navigation menu in the top right corner of the screen the customer can press the “my location” button to set their location, they can drag the map and drop a pin or type in their address/co-ordinates/postcode.	Medium
User can navigate the map by dragging it across their screen.	The user can drag with their thumb or mouse to move the map, they can zoom in and out via reverse pinching their fingers or scrolling their mouse’s scroll wheel.	Medium
Barber profile		

Customer must be able to see where barber is.	There will be a small map window with the selected barbershop's location pin annotating their location. Below the map will be a button "See on Google Maps" that will transport the customer to the barbershops Google Maps pin.	Medium
See photos of other haircuts performed by the barber.	There will be a collage of images that the barber uploads to their profile to show their skills. Some images will be fixed place adverts, and some will be in a gallery widget that the customer can peruse by using arrow buttons to change images.	High
Must be a "go to website" button for user to navigate to barber's own website.	This is a simple linked button widget that will be below the map window. It will say "go to website" and will take the user to the barber's own website.	High
Customer can see barbers rating and reviews.	There will be a star rating visible at the top right of the page above the bio. If customer scrolls down page, they will see the comment section which they will be able to view comments from. The comment section will be a sub window on this page like the map. There will be a scroll bar the user can use to scroll through the comments. Each comment will show customer's profile picture and username as well as the date uploaded.	High
Customer can rate and review barber.	The barber profile page will include a "Rate & Review" button beneath the star rating section. When this is selected a pop up page will appear letting the customer to choose a star rating and optionally write a short text review. After submission the system checks the input and adds the review to the barber profile and then updates the barber's overall rating.	High
Customer can follow barber.	A "Follow" button will be shown under the barber's name and profile picture. When clicked the system will store the barber's ID in the customers account so the barber's post will appear under the "Followed Barbers" filter on the discover page and once followed the button will switch to "Unfollow".	Low
Customer can share barber's profile.	A "Share Profile" button will generate a shareable URL that links directly to the barbers profile and when pressed the system will open the device's native share menu on mobile.	Low
Customer can see barber's specialities.	Under the barber's bio the system will display the barber's specialities that they chose in the dashboard, these will be shown as labelled tags/icons for quick scanning.	Medium
Customer can see when barber is working.	The barber profile will include a "Working Hours" section thats displays the barbers weekly schedule. This information is taken from the	Medium

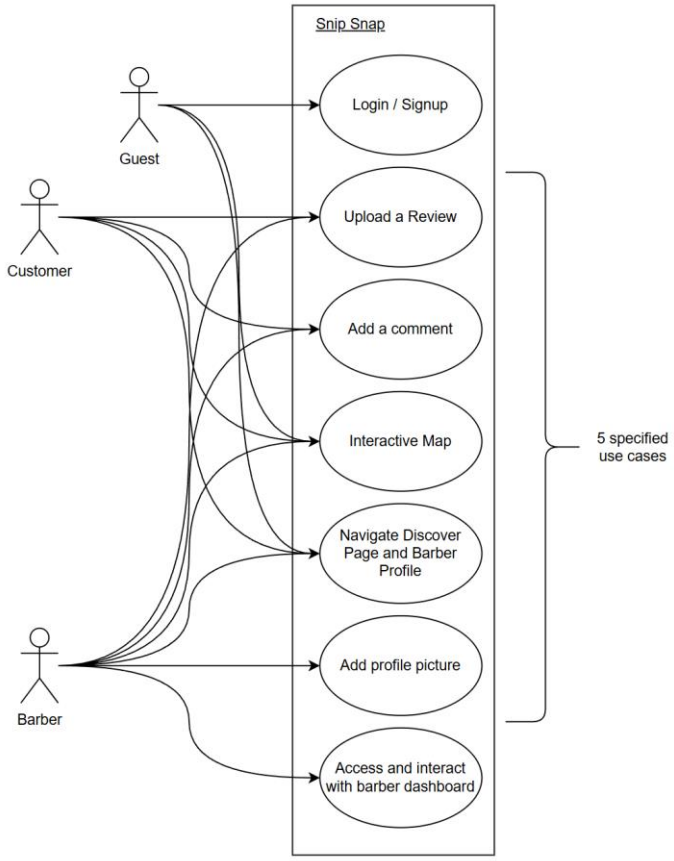
	barber's dashboard where the barber sets their availability.	

System requirements (non-functional)

Non-functional requirements		
Aesthetic	The system UI must have a consistent and easily understandable style for both phone and desktop users. Buttons, text boxes, search bars must all have a similar aesthetic to make the site easy to use.	
Response time	All processes must be executed within 3 seconds to prevent user frustration.	
Security	GDPR laws will be followed to keep user information secure, up to date and relevant.	
Role validation	Users will have roles that restrict their access, e.g. customers cannot access Barber Dashboard because they aren't a barber. Barbers can access any page. Guest users can access same pages as Customer but with restricted access preventing them from submitting reviews and following barbers.	
Database behaviour		
The app will need to store non-volatile data in a database. The key entities will be.	User (with a ROLE of customer or barber), barber profile (linked 1-1 with barber type User), Barbershop, Post, Review, Follow.	High
The database will follow typical database conventions.	Unique PK, referential integrity on FK, Unique email and username, 3rd normal form, sensible/readable naming conventions, enforced data integrity.	High
Indexes and performance.	The database will automatically create indexes on all primary & foreign keys. Also put indexes on attributes used for filtering posts e.g., Post.created_at, BarberProfile.average_rating, BarberProfile.distance. Other indexes will be assigned during testing phase on app prototype.	Medium
Accounts and sessions		
Account creation	The system will allow users to create an account by supplying their email, a unique username and a secure password. The system will validate username and password against set rules (unique, how many special characters etc). The system will prevent creation of duplicate accounts with same email address.	High
Login	The system will authenticate users by taking email and password and then verifying against the stored hash password.	High

Sessions	The system will create a “Session Token” on successful login. The system will associate each request with an authenticated user by looking at their session token.	High
Logout	There should be robust logout mechanism to prevent residual session tokens being left in service.	High
Security		
Protection of database data and API keys	The system will keep access credentials and API keys in server local files. The client must never see them (in the code).	High
Input validation	All input validation will be performed server-side to ensure it matches; client's machine cannot be trusted to do this.	High
Password security	The system will never store passwords in plain text. They must be stored as a hash and then when the user logs in the inputted password will be hashed and the two hashes compared. At no point in time will any password be readable plaintext.	High
Discover page algorithm		
Basic	The system will show posts based on some relevance score. The score will be calculated by the distance, rating, follow and other tags the user enters.	High
Distance	Distance will be calculated by using google maps API and barber shop co-ordinates.	High
Review	Barber’s rating will be calculated by averaging all reviews from their past year.	High

Use case diagram



This diagram shows which users can interact with what. When the user opens the app, they are automatically a Guest. The Guest can login or signup to make an account, alternatively they can bypass the login/signup process and just continue as guest with restricted access to Review and Follow features. Customers can access all the core features except the Barber Dashboard because they're not a barber and login/signup because their already logged in. Barbers can access all core features of the app except for login/signup. Barbers can view discover page to peruse their competition and view their own or other's barber profiles, leave reviews, they may use app like customer if they need to.

System architecture

Snip snap will be implemented as a three-tier client server web application where the client tier will contain the user interface for both barbers and customers, the server component will implement the application logic as a set of services and the data tier has the nonvolatile information in a database. This architecture was chosen to separate presentation, business logic, and data management concerns leading to the system being easier to maintain, grow, and secure.

Client Tier

The client tier groups together all pages and widgets that the end users such as the barbers, customers and guests interact with: the Barber Dashboard Page UI, the login and Signup Page UI, the Discover Page UI, The Interactive Map UI and Widget, the Barber Profile Page UI and the Review Widget UI. All UI elements correspond directly to at least one or more of our user requirements and use cases (for example, "View Discover Feed", "Upload Review"). The client does not access the database directly, instead they invoke operations exposed by the server, as

shown by the function style labels on the arrows. For example, the Discover Page UI calls get_10_posts(all_current_post_PK) on the discover Feed Service, while the Review Widgets calls upload_review(text, reviewer_profile, root) and delete_review() on the Review Service.

Server Tier

The server tier contains the application logic and is divided into different services. All clients request first pass through the account and security service which is responsible for validating login credentials, creating new accounts, checking email uniqueness and validating section tokens, this reinforces the security requirements from the system requirements such as password hashing, input validation and role based access (guest, customer, barber).

The Barber Dashboard Service implements the barber side use cases such as changing profile information, uploading and removing images, editing services and updating links through various functions such as add_gallery_image(), change_location() and add_speciality_tag(). The Barber Profile Service provides read only information to customers, for example get_profile_photos(), get_barber_info() and get_barber_website_link() which are used by the barber profile page UI.

The Discover Feed Service contains the logic for retrieving and filtering posts that may appear on the Discover Page. It implements operations such as add_filter(filter) and get_10_posts(), and will internally apply relevance rules based on distance, ratings, tags and followed barbers.

The Interactive Map Service is responsible for all map based functionality, including retrieving barbershop locations and pins through functions like get_barbershop_pins_near_me(). It shares the external geographic lookups to the Google Maps API connector. Lastly, the Review Service supports the “Upload Review”, “Edit Review” and “Delete Review” use cases by providing corresponding operations and ensuring that only the review owner can edit their reviews.

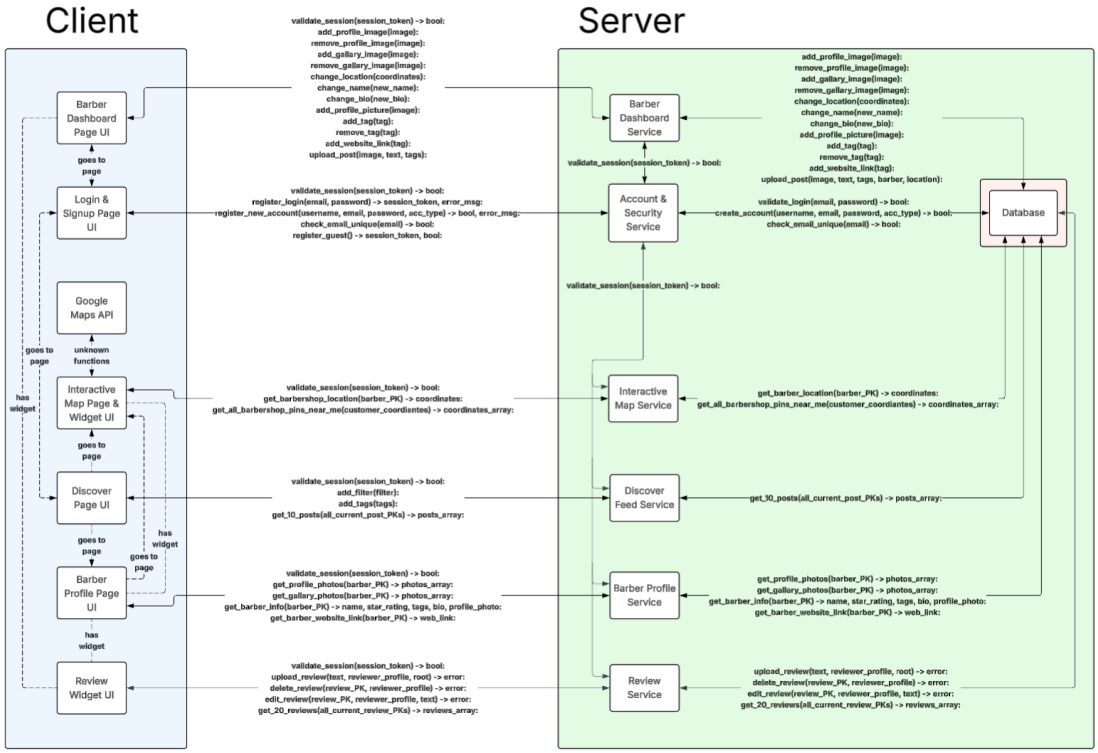
Data tier

The data tier contains a relational database and database access layer where all services interact with the database through this layer instead of issuing SQL Queries directly. The schema will contain tables for Users, Barbershop, Post, Review, and Follow. These entities correspond to the database requirements in the requirements such as storing barber’s posts for the discover feed, persisting customer reviews and ratings, and maintaining follow relationships. The database layer enforces referential integrity, unique email and username constraints, and indexing frequently queried attributes such as distance or creation time.

Alignment with requirements and use cases

Each service in the architecture maps directly to a set of system requirements and use cases. For example, the Discover Feed Service realizes the “View Discover Feed” use case and the discover-page system requirements; the Interactive Map Service realizes the “View Interactive Map” use case; the Barber Dashboard and Profile Services implement the barber-side management use cases; and the Review Service supports “Upload Review”, “Edit Review” and “Delete Review”. This ensures traceability from user requirements, through system requirements, to concrete architectural components.

System diagram

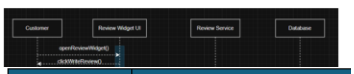


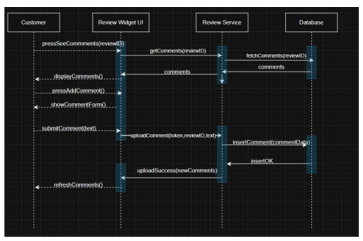
Use Case Definitions and System models

Use Case:	Upload a Review
Actors	Customers, Barbers
Description	Customers or Barbers can write reviews for barbers or barbershops. The reviews can be written in a comment section sub window found on either the barber profile or barbershop profile. Reviews consist of a star rating and text.
Pre-Condition	The review upload function can only be accessed from the review widget. The review widget is a sub-window that can be found on either the barber profile or barbershop page.
Post-Condition	System displays a message to user informing them that the process has completed successfully.
I/O Data	A session token, a star rating, text less than 1000 char
Interaction steps	1. When review sub window is loaded at page instantiation it will add a button called "write review" if the user's session token belongs to a customer. 2. Customer presses "write review" button on reviews sub window 3. The review creation form pops up onto their screen. 4. Customer presses which star rating they want to give (required) 5. Customer writes their review (optional) 6. Customer presses "upload review" 7. The client-side Review Widget UI component sends the data from the form to the server-side Review Service. 8. The review service checks the session token to validate the review as coming from an active customer. 9. Review service sanitises input to prevent SQL injection. 10. Service also checks for and censors profanities with "*s 11. Now the Review Service checks that the review's text is unique if not null. 12. Review is inserted into database. 13. Once DB write is complete the Review Service will return null to Review Widget UI thus confirming there were no problems 14. Review Widget UI displays message "review uploaded successfully!"
Things that can go wrong	1. If the user's session token is not valid the review window will not load. If the token does not belong to a customer, the review window will load but the "write review" button will not. 2. Make sure form outputs data in correct format for Review Service to use 3. If the session ID is invalid just return Null. 4. If SQL injection is detected, then the review service will invalidate the session token thus locking the user out from that session. 5. If text is not unique an error message will be returned to the customer informing them that they need to make a unique comment. 6. If the insert fails for whatever reason the customer will be asked to try again. By the time the data is being sent to the database it should be sanitised and formatted as required.
Valid Case	A customer gives a barber a 4-star rating and a short-written review too.
Error Case	A guest can't even press the "Upload review" button. Another error case would be an invalid input such as more than 1000 characters of text.

Use Case:	Add a comment
Actors	Customer, Barber
Description	Customers or barbers can reply to reviews with comments. This allows barbers to respond to negative reviews, or positive reviews too. Comments are similar to reviews except their root/parent is a review or comment and they don't have star ratings. Review's root/parent can only be the review widget sub window.
Pre-Condition	A review must exist for the user to comment on.
Post-Condition	The new comment is stored in the database and is visible under the review.
I/O Data	
Interaction steps	1. User presses "see comments" button. 2. User presses "add comment" button. 3. The add comment form appears on screen, it prompts the user to enter some text. 4. User writes their comment. 5. User presses "upload comment" button. 6. Data from the form is sent to the server-side Review Service component 7. Comment is inserted into database 8. Review Widget UI is re-loaded to display the new comment. 9. Review Widget UI component displays message "comment uploaded successfully!"
Things that can go wrong	1. There may be no comments, in this case there should be a message saying "no comments" 2. This option won't be available to guests. 3. Make sure dimensions are dynamic, phone and desktop should be different. 4. None 5. None 6. Data must be correct format for the Review Service to work with it. If data isn't in correct format and error message will be returned. Data should be formatted by the Review Widget UI component 7. Hopefully nothing goes wrong, if something doesn't work then user will be prompted to try again. 8. If the barber added a comment the widget would reload with that comment shown, otherwise the comment will be hidden under "see comments" button 9. None, well, dynamic formatting blah blah
Valid Case	Customer presses "add comment," enters text, uploads it, and the comment appears immediately under the review with a success message.
Error Case	Data sent in the wrong format, system returns an error and does not upload the comment.

Use Case:	Interactive Map
Actors	Customers, Guests
Description	The customer views an interactive map showing nearby barbershops. The map displays pins based on the customer's location. Selecting a pin shows barber information and provides a button to navigate to the barber's profile page.
Pre-Condition	- User has a valid session token (guest or customer). - Google Maps widget loads successfully. - User location (Gps/manual entry) is available and valid.
Post-Condition	- Customer has viewed nearby barbershops on the map. - Customer may navigate to a barber profile. - Application may have updates map bounds and refreshed pins.
I/O Data	- Session token - Customer coordinates (latitude/longitude). - Map interactions (pan/zoom). - Google Maps UI centred on customer's location. - Array of barbershop pins with coordinates and barberID. - Popup with barber name, rating and "View Profile" link.
Interaction steps	1. User opens the interactive Map Page UI. 2. UI sends the session token to validate_session(token). 3. UI initialises Google Maps widget and requests customer location (GPS or manual input). 4. UI sends customer coordinates to the Interactive Map Service. 5. Service calls get_all_barbershop_pins_near_me(coords) to get nearby barbershops. 6. Service returns an array of pins (coordinates, barber ID, + basic info). 7. UI renders the map centred on the customer's location and displays pins. 8. If the customer pans/zooms, UI sends updated bounds back to the service and repeats steps 4-7. 9. When a customer clicks a pin, UI displays a popup with barber info and a "View Profile" button. 10. Customer selects "View Profile", UI navigates to the Barber Page using the barber's ID.
Valid Case	- Customer location is obtained successfully. - Nearby barbershops are found. - Pins display correctly. - Customer selects a pin and proceeds to the Barber Profile Page without errors.
Error Case	- An invalid session token leads to a redirection to the login page. - Google Maps widget failing to load leads to manual location entry being entered and if that is unavailable the map cannot be initialised. - Invalid or unresolved coordinates leads to users having to re-enter their location. - If malformed coordinates are sent to the service an error is returned which is then displayed by the UI. - If there are no barbers in the area the UI displays "No barbers found in this area". - Excessive panning causes rate limit therefore the UI will throttle map refresh calls. - Popup rendering issues on mobile, this can be fixed by having the UI adjusting placement.





Use Case:	Navigate discover page & view
Actors	Customer
Description	The customer opens the Disco app shows a list of available ba selects a barber, the system lo
Pre-Condition	Customer must be logged in w Discover service must be avail
Post-Condition	The customer can see the cho
I/O Data	
Interaction steps	1. Customer opens the Discover page in the app. 2. The app checks the customer's login session using the session token. 3. The app requests a list of barber posts from the Discover service. 4. The Discover page shows the returned barber posts.

Use Case:	Add profile picture
Actors	Barbers
Description	Barbers are able to add a profile picture of which customers can view and click on. This profile picture can be seen on both the barber's profile and on the explore page when a customer comes across the barber
Pre-Condition	The barber must have access tot their profile page
Post-Condition	A new profile picture is stored and shown on the barber's profile and on the Explore page.
I/O Data	
Interaction steps	1. User must have valid session token, in this case barber 2. The barber opens their profile 3. The barber clicks on an 'edit profile button' displayed on their profile 4. The barber is then given an array of different actions to do with their profile 5. The barber clicks on the 'add profile picture' button 6. The barber is then prompted to add an image

Implementation method

Our implementation method takes inspiration from concepts learned in other modules of this course. Since our app is a social media like platform, we need the app to be as *usable* as possible. To make our app super usable, we must engineer the app with a focus on usability first, then focus on functionality. Our implementation process will be as follows.
First we will draw out exactly what each page, widget and form will look like on paper during group meetings. Doing this as a group will allow us to copy each other as we go and thus develop a consistent style across our User Interface, this will allow us to fulfil the system requirement for consistent aesthetic. Once we have what the website will look like on phone and desktop planned out, we can start assigning pages, widgets and forms to specific components as seen in the Client section of our system diagram.
Now with all the User Interface elements and their respective components planned out, we can start designing the backend systems to fit the front end. If we did the backend systems before the front-end user interface that would be like building a factory except you don't know what you're making, in other words, quite silly. Each system in the front-end will need to be linked to a respective service from the backend. For example the Discover Page UI will need to be linked to the Discover Feed Service. By designing the UI first and then designing the backend to fit, each end can be programmed by separate teams simultaneously and then joined together at

the end with a simple API call. This is a workflow we were taught in our Usability Engineering Module and is widely used for its effectiveness.

When we develop the app with code, we will use tools to help us. One of the best tools for software engineering team development is GitHub, it's integration with through terminals to the online repository and its version control features with push/pull requests etc makes GitHub THE software development hub for most if not all software engineers. Now, regarding the tech stack we will be implementing with we are going to use python and flask for the server-side logic because Flask is good for beginners, has good support for python and python is what everyone on the team can program. For the client-side front end we will be making a webapp with HTML5, JavaScript and React. HTML5 for basic website structure, JavaScript and React for website interactivity, React isn't a programming language, it's a library of code that developers use so that they don't need to program complex UI elements and instead can simply add them as object with modularity and consistent styling across the webapp. Finally for the database we will be using PostgreSQL as that is the only programming language for databases that we know of.

Testing method

To test the snip snap app's functionality, we will perform a few different testing methods. By using a few different methods, we can get a more accurate understanding of the systems functionality.
First, we will test the system as we go, this is testing during implementation. If I'm making the discover page UI and I want to see if my filter drop down list is working ill just run the code and click the list. This method is useful for testing smaller systems like drop down lists but is not good at spotting bugs on a zoomed out system scale.
For spotting bugs, we will get a participant to do their best at breaking our app. Since we made the app, we will only interact with it in the way we intend it to be interacted with, this makes us unfit for the role of bug testing.
Lastly, we need to test that we have built a system that meets our system requirements. To test this, we simply need to copy the requirements table from above and show where that requirement has been met.

Critical Analysis

Leadership

Over the course of the project, leadership was not fixed. For example, at the beginning of our project, one group member took the initiative to come up with a design and outline to the rest of the group what kind of ideas should be included. During these early stages, another group member outlined how the workload should be split, ensuring everyone agreed. The style of leadership was a democratic one, as any given group member who was aware of the tasks at hand at each stage of the project would take the initiative to update the rest of the group. This approach allowed us to avoid confusion and ensured each member was aware of their responsibilities. However, a challenge with this approach was that leadership would sometimes be inconsistent, as no single member had a definitive role of leader. To combat this,

one member of our group would take on the role of leader and ensure we are meeting deadlines. This experience highlighted the importance of having a clearly defined leadership at key stages of a project, as while a shared leadership encourages collaboration, the lack of a consistent leader caused multiple delays in decision making. As a result, we learnt that even in a democratic team, assigning a temporary leader for specific tasks can improve efficiency and accountability.

Progress monitoring

We would monitor our progress through weekly group meetings, often held on a Thursday morning, and regular communication through Discord. In the meetings, we would discuss how much work has been completed and what the next steps are. These meetings were effective in knowing whether the project was moving forward at the desired pace. Despite this, there were occasions when tasks were getting completed slower than expected. In order to combat this issue, the group would reconvene and adjust deadlines and workload when needed to ensure that the work was completed. For example, interviewing barbers took longer than expected, as not many barbers agreed to take part in interviews, and looking for other barbershops to interview took too long. From this we learnt that we should have prioritised the task earlier to account for the delay of having to find a barber that's willing to be interviewed for our project, as many could be unwilling.

Conflict resolution

There was minimal conflict within the group; however, minor disagreements did arise regarding task distribution and the level of detail required in certain sections of the project. These conflicts would be resolved through open discussion on our discord chat, where each member would give their opinion and reasoning on the conflict at hand. This approach proved to be effective, as it helped prevent the conflicts from escalating further and ensured decisions were based on fairness and project requirements rather than individual preference. Through this we learned that addressing disagreements early and openly prevent misunderstandings and strengthens teamwork.

Links

Survey Link : Interveiws

Group Contributions

George	Contributions:
	• Coursework Doc
	  o Wrote elicitation method section of coursework doc
	  o Also wrote Analysis method paragraph
	• User requirements
	  o Discover page
	  o Barber profile
	  o Both login pages
	• System requirements
	  o User must see an “endless” scroll of photos posted by barbers.
	  o Customer must be able to filter their feed to get a specific service that they want.
	  o Filter by followed barbers.
	  o Customer can click on image to be taken to barber’s profile.
	  o Sign-up process
	  o Barber account creation
	  o Customer account creation
	  o They can edit their profile information.
	  o They can specify their exact services and skills.
	  o They can upload images for their profile
	  o User can specify their location.
	  o User can navigate the map by dragging it across their screen.
	  o System requirement sections:
	  ♣ Database behaviour
	  ♣ Accounts and sessions
	  ♣ Security

	♣ Discover page algorithem ♣ Non-functional requirements • System diagram o Made the first draft, Cyrus did second draft and then I did a third draft. • Use Case Definitions o Upload a review o Add a comment • Interviews o Initial barber interview o Initial customer interview o 1 more customer interview o 4 more barber interviews • Analysis o Did the entire analysis of all interviews myself (was quite easy actually) • Implementation section • Testing section • Use case diagram • Was team leader from week after project start until 03/12/2025 Overall Percentage: 14%
UP2212962	- System Requirement - Barber Interview - Customer Interview - User Requirement - Use Case Definitions: Navigate discover page & view barber profile - Created sequence diagram - Wrote up system diagram and explanation - Implementation - Testing - Critical Analysis - Coursework Document Overall Percentage:14%
UP2275562	- System Requirement - Barber Interview - Customer Interview - User Requirement - Use Case Definitions

	- Wrote up system diagram and explanation - Implementation - Testing - Critical Analysis - Coursework Document Overall Percentage: 14%
UP2273554	- System Requirement - Barber Interview - Customer Interview - Created use case diagram - Wrote up system diagram explanation - Created a second version of system diagram - Created sequence diagrams for upload a review - Created sequence diagrams for add a comment - Created sequence diagrams for navigate discover page & view barber profile - Created sequence diagrams for add profile picture - Implementation - Testing - Critical Analysis - Coursework Document Overall Percentage: 14%
UP2264290	- System Requirement - Barber Interview - Customer Interview - User Requirement - Use Case Definitions - Wrote up system diagram and explanation - Implementation - Testing - Critical Analysis - Coursework Document Overall Percentage: 14%
Up2272567	- System Requirement - Barber Interview - Customer Interview

	- User Requirement - Use Case Definitions - Wrote up system diagram and explanation - Implementation - Testing - Critical Analysis - Coursework Document Overall Percentage:14%
UP2306576	- System Requirement - Barber Interview - Customer Interview - User Requirement - Use Case Definitions - Wrote up system diagram and explanation - Implementation - Testing - Critical Analysis - Coursework Document Overall Percentage: 14%